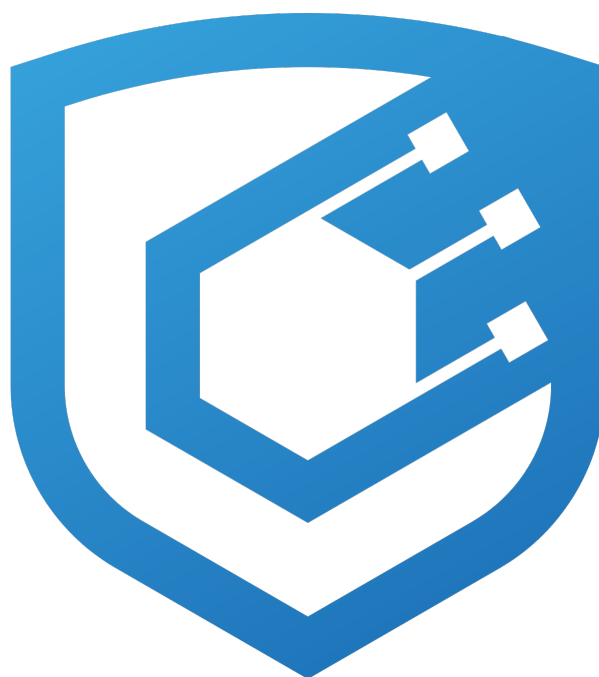




Securitize Solana Whitelister Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Farouk](#)

[JesJupyter](#)

[Ctrus](#)

September 17, 2026

Contents

1	About Cyfrin	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	3
6	Executive Summary	3
7	Findings	5
7.1	Medium Risk	5
7.1.1	Improper investor-to-identity validation allows arbitrary wallet attachment to any investor identity	5
7.2	Low Risk	6
7.2.1	Investor's token balance is not checked prior to registration	6
7.3	Informational	7
7.3.1	VaultRegistered event omits investor signer when signature is required	7
7.3.2	set_require_investor_signature allows setting to current value, emitting misleading event	7
7.3.3	Dead Error Variants and Documentation Inconsistency for Signature Requirements	7
7.3.4	Permissionless Initialize and Counter Saturation Edge Case	8
7.3.5	Insufficient validation on vaults	9
7.3.6	Explicitly reject Pubkey::default() for vault_wallet or the investor_wallet	9
7.4	Gas Optimization	10
7.4.1	Redundant Zero-Address Check in revoke_operator	10

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

The VaultRegistrar is a Solana Anchor program that serves as a compliance gatekeeper between DeFi protocols and the Securitize DS Token ecosystem. Its primary function is to enable authorized operators (DeFi protocols) to atomically register vault addresses under an investor's existing DS Token identity, allowing those vaults to interact with regulated real-world asset tokens without granting operators broad exchange permissions.

The core mechanism centers on a CPI chain that spans all four of Solana's allowed CPI depth levels: a DeFi protocol calls the VaultRegistrar, which in turn calls into the rwa-rbac program, which calls the Identity Registry, which finally invokes the System Program. When an operator calls `register_vault`, the VaultRegistrar PDA signs a CPI to `rwa_rbac::attach_wallet_to_identity`, binding a new vault wallet to an existing investor identity in the Identity Registry. An optional `require_investor_signature` flag, when enabled, requires the investor to co-sign the transaction, providing an additional layer of consent verification beyond operator authorization alone.

The system is bounded by several key constraints. Each VaultRegistrar instance is permanently tied to a single asset mint. The admin is mutable and may be updated after initialization. A maximum of ten operators may be registered per instance. The admin controls pausing, operator management, admin updates, and the investor signature requirement toggle. The VaultRegistrar PDA itself must be explicitly granted the "exchange" role (user role id=1) in the DS Token's access controller before it can perform any meaningful action. The creation of a VaultRegistrar instance is gated by the `ALLOWED_INITIALIZERS` constants.

The architecture consists of a single in-scope program, the VaultRegistrar, with supporting state accounts: a global counter PDA for generating unique registrar IDs and per-instance VaultRegistrarState PDAs storing admin, operators, pause state, and configuration. The program depends on external, out-of-scope programs (rwa-rbac and Identity Registry) for the actual identity attachment and compliance enforcement.

For the system to be safe, the CPI chain must correctly validate all account inputs at each level, the VaultRegistrar's exchange role must be tightly controlled by the DS Token admin, and the investor signature mechanism must authentically bind consent to the identity being modified.

5 Audit Scope

The audit scope was limited to:

```
programs/vault-registrar/src/instructions/admin/add_operator.rs
programs/vault-registrar/src/instructions/admin/initialize.rs
programs/vault-registrar/src/instructions/admin/is_admin.rs
programs/vault-registrar/src/instructions/admin/is_operator.rs
programs/vault-registrar/src/instructions/admin/pause.rs
programs/vault-registrar/src/instructions/admin/revoke_operator.rs
programs/vault-registrar/src/instructions/admin/set_require_investor_signature.rs
programs/vault-registrar/src/instructions/admin/unpause.rs
programs/vault-registrar/src/instructions/register_vault.rs
programs/vault-registrar/src/states/vault_registrar_counter.rs
programs/vault-registrar/src/states/vault_registrar_state.rs
programs/vault-registrar/src/constants.rs
programs/vault-registrar/src/lib.rs
```

6 Executive Summary

Over the course of 2 days, the Cyfrin team conducted an audit on the [Securitize Solana Whitelister](#) smart contracts provided by [Securitize](#). In this period, a total of 9 issues were found.

The VaultRegistrar is a Solana Anchor compliance program that enables authorized DeFi protocol operators to atomically register vault addresses under investor identities in the Securitize DS Token ecosystem. The security model relies on correct CPI chain validation across four depth levels, proper exchange role assignment by the DS Token admin, and an optional investor signature requirement to ensure investor consent.

Our review focused on the `register_vault` instruction and its CPI data integrity, the investor signature enforcement mechanism, operator authorization controls, the then-permissionless `initialize` flow, and admin operations including pause and signature requirement toggling.

The audit identified 9 findings: 1 Medium, 1 Low, 5 Informational, and 1 Gas Optimization. No critical vulnerabilities were discovered. The medium severity finding relates to improper investor-to-identity validation in the `register_vault` instruction, where the investor signature check does not verify that the signing wallet actually belongs to the identity account being modified. This allows a privileged operator to satisfy the signature requirement with any wallet while attaching a vault to an arbitrary investor identity, undermining the intended consent mechanism even when `require_investor_signature` is enabled. The low severity finding identifies a missing investor token balance check that is present in the EVM equivalent, allowing registration for investors who may not have completed the full onboarding process.

Additional findings address EVM-Solana parity gaps including missing vault re-registration checks and absent zero-address validation for vault and investor wallets, dead error variants and documentation inconsistencies around vault signature requirements, a missing investor field in the `VaultRegistered` event, a no-op guard absence on the signature requirement setter, and the permissionless `initialize` design with its theoretical counter saturation edge case. The gas optimization item identifies a redundant zero-address check in operator revocation.

Overall, the codebase demonstrates a well-structured architecture with clear separation of concerns and appropriate use of Anchor's constraint system.

The initial review took place from March 6th to March 9th, 2026. Most findings were addressed by the Securitize team shortly afterward. On September 16th, 2026, the team submitted a further change that restricted `initialize` to `ALLOWED_INITIALIZERS`, resolving the permissionless `initialize` finding, and added a `change_admin` instruction (located in `instructions/admin/change_admin.rs`) to update the admin address. Cyfrin reviewed and confirmed both changes. `change_admin` was introduced after the original engagement and is not part of the original audit scope.

Summary

Project Name	Securitize Solana Whitelister
Repository	bc-solana-whitelister
Commit	edb0c4a19cb4...
Fix Commit	44aef5dd6e0f...
Audit Timeline	Mar 6th - Mar 9th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	1
Low Risk	1
Informational	6
Gas Optimizations	1
Total Issues	9

Summary of Findings

[M-1] Improper investor-to-identity validation allows arbitrary wallet attachment to any investor identity	Resolved
[L-1] Investor's token balance is not checked prior to registration	Resolved
[I-1] VaultRegistered event omits investor signer when signature is required	Resolved
[I-2] set_require_investor_signature allows setting to current value, emitting misleading event	Resolved
[I-3] Dead Error Variants and Documentation Inconsistency for Signature Requirements	Resolved
[I-4] Permissionless Initialize and Counter Saturation Edge Case	Resolved
[I-5] Insufficient validation on vaults	Resolved
[I-6] Explicitly reject Pubkey::default() for vault_wallet or the investor_wallet	Resolved
[G-1] Redundant Zero-Address Check in revoke_operator	Resolved

7 Findings

7.1 Medium Risk

7.1.1 Improper investor-to-identity validation allows arbitrary wallet attachment to any investor identity

Description: `register_vault` is intended to protect wallet attachment with `require_investor_signature`, but the current implementation does not authenticate the investor whose identity is being modified.

The Solana flow only checks that:

1. `existing_investor_wallet` signed the transaction, and
2. `existing_investor_wallet_identity` is the PDA derived from that wallet.

It does **not** verify that `existing_investor_wallet_identity` is a valid `WalletIdentity` account bound to the same `identity_account` that is later passed into `attach_wallet_to_identity`. As a result, there is no on-chain binding between the purported “existing investor” and the identity actually being mutated.

This means an operator can satisfy the signer check with any wallet they control and still attach an attacker-controlled vault/wallet to an arbitrary victim `identity_account`. The downstream RBAC / Identity Registry CPI only uses the target `identity_account` and the new wallet being attached, so the “existing investor” accounts do not constrain the identity being modified.

`require_investor_signature = false` is obviously unsafe, but even `require_investor_signature = true` is still bypassable because the signer is not required to belong to the target investor identity. Unlike the EVM implementation, the Solana flow does not verify that the passed investor wallet actually corresponds to the investor whose identity is being updated.

Impact: A privileged operator can impersonate any investor identity by attaching an attacker-controlled wallet or vault to that identity.

Once attached, downstream DS-token infrastructure treats the attacker-controlled wallet as the victim identity for normal issuance, transfer, and compliance checks. This allows the attacker to:

1. receive and transfer DS tokens under the victim’s compliance profile;
2. bypass country / level-based restrictions that would apply to the attacker’s real identity;
3. pollute the victim’s identity-wide tracker state and potentially disrupt the real investor’s future transfers.

In integrated deployments, the impact becomes more severe. In the off ramp, `redeem_ds_token` accepts a valid `wallet_identity -> identity_account` binding as sufficient investor authorization. Therefore, once an attacker-controlled wallet has been fraudulently attached to a victim identity, the attacker can redeem DS tokens from that wallet into attacker-owned liquidity tokens through the off-ramp.

The worst case is:

1. attach attacker wallet to a privileged / unrestricted victim identity;
2. route DS tokens into that forged wallet through ordinary issuance or transfer flows;
3. redeem those DS tokens through the off-ramp into liquidity tokens controlled by the attacker.

Recommended Mitigation: Require the identity account to be owned by the `existing_investor_wallet`.

Securitize: Fixed in [c7e5ff7](#).

Cyfrin: Verified.

7.2 Low Risk

7.2.1 Investor's token balance is not checked prior to registration

Description: The Solana implementation of the VaultRegistrar does not enforce the investor token balance check present in the EVM equivalent. In the EVM protocol, the contract verifies that the investor holds the relevant asset (`IERC20(token).balanceOf(investorWalletAddress) > 0`) before allowing the registration of a new vault. The current Solana `register_vault_handler` only checks for the presence of the `existing_investor_wallet` and its identity PDA but fails to verify if the wallet actually holds any tokens of the `asset_mint`.

Impact: If the investor doesn't hold tokens it means maybe there is something that has not been complete from KYC or on boarding standpoint, and such investors must be prevented from registration.

Recommended Mitigation: Implement the evm equivalent check.

Securitize: Fixed in [f16257a](#).

Cyfrin: Verified.

7.3 Informational

7.3.1 VaultRegistered event omits investor signer when signature is required

Description: The VaultRegistered event at register_vault.rs:138-144 emits registrar, caller, vault, asset_mint, and identity_account, but does not include the existing_investor_wallet pubkey when require_investor_signature is enabled.

```
emit!(VaultRegistered {
    registrar: ctx.accounts.vault_registrar_state.key(),
    caller: ctx.accounts.caller.key(),
    vault: ctx.accounts.vault_wallet.key(),
    asset_mint: ctx.accounts.asset_mint.key(),
    identity_account: ctx.accounts.identity_account.key(),
});
```

Off-chain monitoring systems cannot determine which investor provided consent from the event alone.

Recommended Mitigation: Add an optional investor field to the VaultRegistered event.

Securitize: Fixed in [17bb95e](#).

Cyfrin: Verified.

7.3.2 set_require_investor_signature allows setting to current value, emitting misleading event

Description: set_require_investor_signature unconditionally sets the flag and emits a RequireInvestorSignatureUpdated event, even when the new value equals the current value. This produces misleading events suggesting a configuration change occurred when none did.

Recommended Mitigation: Add a guard to prevent no-op updates:

```
require!(
    state.require_investor_signature != require_investor_signature,
    VaultRegistrarError::AlreadySet // new error code
);
```

Securitize: Fixed in [3c93d56](#).

Cyfrin: Verified.

7.3.3 Dead Error Variants and Documentation Inconsistency for Signature Requirements

Description: The error variants VaultSignatureRequired and InvestorSignatureRequired are defined in errors.rs but never used in the program logic. Additionally, the README references require_investor_and_vault_signatures and states that "both the investor and vault keypairs must sign," whereas the actual state field is require_investor_signature and vault signature is never enforced, creating documentation inconsistency.

In programs/vault-registrar/src/errors.rs:

```
#[msg("Vault signature required: vault wallet must sign the transaction")]
VaultSignatureRequired,

#[msg("Investor signature required: existing_investor_wallet must sign the transaction")]
InvestorSignatureRequired,
```

In register_vault, when require_investor_signature is true, only InvestorAccountsRequired is used:

```
if vault_registrar_state.require_investor_signature {
    require!(
        ctx.accounts.existing_investor_wallet.is_some()
        && ctx.accounts.existing_investor_wallet_identity.is_some(),
        VaultRegistrarError::InvestorAccountsRequired
    )
}
```

```
);
}
```

`VaultSignatureRequired` and `InvestorSignatureRequired` are never referenced in the program, so they are dead code.

Also,

When `require_investor_and_vault_signatures` is true, pass `investorKp` and `existingInvestor-WalletIdentity`:

When `require_investor_and_vault_signatures` is true on the registrar, both the investor and vault keypairs must sign the transaction. Use `--investor-wallet-path` and `--vault-keypair-path` in that case.

The actual state field is `require_investor_signature` (singular), and `vault_wallet` is an `UncheckedAccount`—the program never enforces vault signature. Only the investor signature is enforced indirectly via `existing_investor_wallet: Option<Signer<'info>>` when the account is provided.

Impact:

- **Dead code:** Increases maintenance burden and may mislead future developers into believing vault signature verification exists.
- **Documentation mismatch:** The incorrect field name and the claim that "both" must sign can cause integration errors or incorrect expectations.

Recommended Mitigation: Remove unused error variants and Fix README if needed.

Securitize: Fixed in [2251615](#).

Cyfrin: Verified.

7.3.4 Permissionless Initialize and Counter Saturation Edge Case

Description: There is no access control on `initialize`. Any payer can create a `VaultRegistrarState` with any `admin` (who must sign) and any `asset_mint`:

```
#[derive(Accounts)]
pub struct Initialize<'info> {
    #[account(mut)]
    pub payer: Signer<'info>,

    pub admin: Signer<'info>,
    // ...
    pub asset_mint: InterfaceAccount<'info, anchor_spl::token_interface::Mint>,
    // ...
}
```

This allows arbitrary actors to create registrars with arbitrary configurations, which may pollute the namespace and complicate discovery of legitimate registrars.

Additionally, when the global counter reaches `u64::MAX`, `saturating_add(1)` saturates and subsequent `initialize` calls will fail because the PDA for that id already exists.

```
ctx.accounts.vault_registrar_counter.set_inner(VaultRegistrarCounter {
    count: id.saturating_add(1),
    bump: ctx.bumps.vault_registrar_counter,
});
```

When `count` reaches `u64::MAX`, it saturates and stays at `u64::MAX`. The next `initialize` call will attempt to create a `vault_registrar_state` with seeds `[VAULT_REGISTRAR_STATE_SEED, u64::MAX.to_le_bytes()]`, but that PDA already exists from the previous successful init. The `init` constraint will fail because the account cannot be created twice.

Impact:

- **Permissionless:** Spam or low-quality registrars may be created, but no direct security impact since each registrar is isolated.
- **Counter saturation:** Theoretically prevents new registrars once `u64::MAX` is reached. Reaching this value is economically infeasible.

Recommended Mitigation: If this intentional, document the permissionless design as intentional.

Securitize: Fixed in commits [b7ebcf17](#) and [4a09894](#).

Cyfrin: Verified.

7.3.5 Insufficient validation on vaults

Description: The Solana implementation lacks the validation checks present in the EVM `VaultRegistrar` designed to verify if a vault has already been registered. In the EVM version, before a vault wallet is added, the protocol queries if the vault is already associated with an identity. If it is linked to a different investor, it purposefully reverts with `VaultBelongsToDifferentInvestor`; if it is linked to the same investor, it reverts with `VaultAlreadyRegistered`.

Currently, the Solana contract calls `rwa_rbac::cpi::attach_wallet_to_identity` directly. Our Solana flow calls the CPI directly without this pre-check., which breaks parity with the EVM architecture and sacrifices granular error handling.

Recommended Mitigation: Implement proper checks just like its evm equivalent.

Securitize: Fixed in [8e3b5f8](#).

Cyfrin: Verified.

7.3.6 Explicitly reject `Pubkey::default()` for `vault_wallet` or the `investor_wallet`

Description: EVM implementation uses `notZeroAddress` for both `vaultAddress` and `investorWalletAddress` as a zero address check. We don't currently explicitly reject `Pubkey::default()` for `vault_wallet` or the `investor_wallet` in solana implementation. It is advised to add this sanity check prior to cpi.

Impact: `vault_wallet` or the `investor_wallet` with default pubkeys can be passed.

Recommended Mitigation: Reject default keys when supplied.

Securitize: Fixed in [e4f82ff](#).

Cyfrin: Verified.

7.4 Gas Optimization

7.4.1 Redundant Zero-Address Check in `revoke_operator`

Description: The `operator != Pubkey::default()` check in `revoke_operator_handler` is redundant. Since `add_operator_handler` already prevents the zero address from being added to the operators list, it can never be present when revoking. The subsequent `state.operators.contains(&operator)` check will always fail for `Pubkey::default()` with `OperatorNotFound`, yielding the same outcome.

In `revoke_operator_handler`:

```
require!(
    operator != Pubkey::default(),
    VaultRegistrarError::InvalidOperator
);

require!(
    state.operators.contains(&operator),
    VaultRegistrarError::OperatorNotFound
);
```

In `add_operator_handler`, the zero address is already rejected:

```
require!(
    new_operator != Pubkey::default(),
    VaultRegistrarError::InvalidOperator
);
// ...
state.operators.push(new_operator);
```

Therefore, `Pubkey::default()` can never exist in `state.operators`. For `operator == Pubkey::default()`, `contains()` will return false and the handler will revert with `OperatorNotFound` regardless of the first check.

Impact:

- Redundant code increases maintenance burden.

Recommended Mitigation: Remove the redundant `operator != Pubkey::default()` check from `revoke_operator_handler`. Rely on `state.operators.contains(&operator)` to reject invalid operators, including the zero address.

Securitize: Fixed in [00d1f81](#).

Cyfrin: Verified.